in the works, so my dreams have some concrete foundation.

It is extremely difficult for a group of 200 to make plans and stick to them. Some of us in the committee try, though.

Q What are all the scheduled features in the C++17 standard that will advance the present programming experience?

Compared to C++11 and C++14, C++17 does not offer anything that will fundamentally change the way you program. It does not change the way you think about constructing a program. Thus, by my usual definition, it is a minor update. It does, however, offer a little for everybody. Importantly, most C++17 features are already shipping in the major implementations; I suspect these implementations will all be feature-complete before 2017 is out.

What matters with the new features is how they can be used to write better code, by which I mean code that more directly expresses its intent and runs faster using fewer resources.

Q Spending hours on the code is not an easy task for young programmers. What valuable tips can they get from you that will help them work comfortably on new projects?

The code is where ideas turn into practical results. We cannot just write papers or manuals. In fact, we must produce code that actually works.

Before adding to a code base, you have to understand some of it. You can get a general understanding of some code top-down, but to be able to make a change (such as to fix a bug) you need to understand the code inside-out, particularly what affects this line of code and in turn, what this line of code affects? Anything that helps read the code and navigate through it, helps. Linear reading or trying to understand everything just does not work. It is not feasible for large code bases and inefficient for small ones.

So, look for good code navigation tools like IDEs and, of course, hope for helpful colleagues who will spend time introducing you to the code base and its associated tools (debuggers, build systems and test suites).

If you want to use C++, take care to learn it well. In particular, learn to use modern C++ well. Do not just repeat the errors of the past. You can write so much better C++11 than you could write in the styles of the 1990s.

Q Finally, where do you see the programming world in the near future?

Hopefully, it will be easier to read, write and maintain code. I do not want to write science fiction, so I will not go into details. But I expect that C++ and I will make a significant positive contribution to that future. END